Using Answer Set Programming to Solve the Automated Warehouse Scenario (CSE 579)

Maclay Teefey
School of Computing and Augmented Intelligence
Arizona State University
mjteefey@asu.edu

Problem Statement

The Automated Warehouse Scenario, created by Gebser and Obermeier (2019) as part of the ASP Challenge 2019, illustrates a real-world scenario to be solved using Answer Set Programming. The scenario proposed describes a retail warehouse system maintained by robot movers. This reflects an increasing number of partially or fully autonomous robots helping to move items across warehouses, like Amazon's Hercules robots (Amazon Staff, 2023). In this scenario, the warehouse is defined as a rectangular grid containing flat robots, shelves, highways, and picking stations. The warehouse takes in a set number of orders that require distinct products to be delivered at varying quantities. The products are stored on shelving units and require robots to lift and carry shelves to picking stations for the products to be delivered. To provide room for movement, highways are designated which do not allow for shelves to be placed. For this scenario the robots are flat, which allows them to move anywhere in the grid. The robots however have three major limitations for their movement: they can only move one tile in the cardinal directions, they cannot collide with other robots, and they cannot occupy the same space as another robot. Each robot can only do five possible actions: move, pickup a shelf, put down a shelf, deliver, or do nothing. Each robot can work concurrently, but each robot can only do one action at a time. The actions each robot takes are logged in an action plan with time represented through steps counting upwards. An action plan is considered a valid solution for this scenario if every order is fulfilled by the robots while not violating the constraints imposed explicitly or implicitly by the scenario. The goal of this scenario is to find the action plans that minimize the number of steps and time taken to fulfill the required orders.

Project Background

Knowledge Representation and Reasoning (KRR) is a subfield of Artificial Intelligence which is focused on communicating knowledge and encoding knowledge for computers to make decisions. Knowledge Representation and Reasoning is used in fields of robotics, computer vision, machine learning, and even allows to quickly display domain knowledge when using Google search (Singhal, 2012). The most common way of encoding knowledge in KRR is in the form of formal languages and logic systems, including Propositional Logic and First-Order Logic. However certain representations of knowledge, including First-Order Logic, do not allow algorithms to know if the formulation of knowledge is satisfiable.

Answer Set Programming (ASP) avoids this issue through using rules and constraints to determine if there are stable solutions to the problem. These constraints are expressed through logical statements that must be satisfied for the program to have a solution. For example, to prevent a robot from picking up a shelf when it was already carrying a shelf, I wrote the constraint in Formula 1.

Formula 1: Robots Only Hold One Shelf

```
:- pickUpShelf(RI,S1,T), carrying(RI,S2,T).
```

This constraint states that at time step T, a robot RI that is holding shelf S2 cannot pick up shelf S1 at the same time step T. Any action plan that includes a robot carrying multiple shelves will not satisfy the program and will be automatically eliminated by the solvers for the program.

The programming language I used for this project is Clingo and its solver of ASP. Clingo allows for many features required for this project, including the specifications of constants and a goal of minimizing a variable.

Scenario Description

The automated warehouse scenario adapts the idea of directing robots in a retail warehouse using ground rules specified in five test instances. The warehouse is defined using a 4x4 grid. Each file specifies the initial locations of the robots, shelves, highways, and picking stations. Along with the location of all possible objects on the grid, the orders and products are defined. Each instance specified the quantity of products for each order, how many of each product are stored on each shelf, and which picking locations have orders assigned to it. The initial configuration of the first test instance is displayed in Figure 1. All of the requirements for an action plan to be valid are determined by code created by the developer.

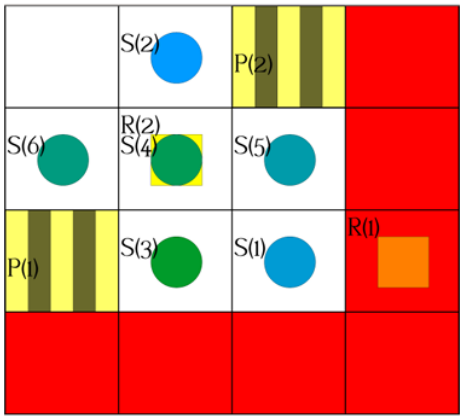

Figure 1: The initial configuration of test instance file "inst1.asp". The robots are represented using squares. The highways are represented using red cells. The shelves are represented using circles. The picking stations are represented using striped cells.

Solution

In order to solve this scenario, I have created an answer set programming file containing instructions to select the set of actions to deliver all of the orders within certain constraints. My solution to this challenge is structured using the Generate-Define-Test method described by Joohyung Lee in his lectures on Answer Set Programming (2026) and provides a useful segmentation of how to gather and select the right action plans. Due to Answer Set Programming being a type of declarative programming, the way of directing solutions is through allowing all possible models to be allowed, a "search space", and then removing models invalid to a problem through problem specific constraints. The Generate-Define-Test structure segments off this development pattern into a generate, define, and test stages.

Generate Stage

The goal of the generate stage is to generate all possible scenarios. I accomplished this through specifying the actions that can be taken. As shown in Formula 2, for the move action I specified using the structure of a choice rule that the action of moving can be in the form of four possible directions: moving up, down, left, or right. A choice rule is a rule structure that allows models to "choose" any of the possible pairs of values to be considered valid.

Formula 2: Movement

```
move(0,1; 0,-1; -1,0; 1,0).
```

In this stage I define the actions robotMove, pickUpShelf, putDownShelf, and deliver with the corresponding objects, like the shelf being picked up and the robot picking up the shelf for the action pickUpShelf. Then I specify the output pattern specified in the scenario instructions, as shown in Formula 3.

Formula 3: The Action pickUpShelf

```
occurs(object(robot,R),pickup,T) :- pickUpShelf(R,_,T).
```

Define Stage

In the define stage, the fluents are defined. Fluents are statements that depend on the state of the program. For this case, the most common aspects that change are the coordinates of the object, and time itself changing. For this program, we want actions to change aspects of the state. Therefore, we define which objects that can change and the actions that can change them. The objects that can change includes: robots, shelves, products, and orders.

To help with act of altering a changing object, state transition statements are created to help specify that an object is being acted upon. This includes robotAt, carrying, shelfAt, orderAt, and productOn.

To prevent objects from being acted upon, these state transition variables are assigned inertial laws. These state variables will continue to be active or inactive for the relevant changing objects as long as actions pertaining to the state transformation statements are not done. For example, a robot will continue to carry a shelf until the action putDownShelf is completed. This rule is shown in Formula 4.

Formula 4: The Inertia Law for Carrying

```
carrying(RI,SI,T+1) :- carrying(RI,SI,T), not putDownShelf(RI,SI,T), T<n.
```

Test Stage

The test stage is where constraints are specified to remove invalid scenario solutions. These constraints can be explicitly specified in the scenario instructions and implicitly

assumed as the aspects of the scenario are described in the previous stages. An example of an implicit constraint is shown in Formula 5.

Formula 5: Putting Down Shelf Constraint

```
:- 2{putDownShelf(R,S,T) : robot(R)}, shelf(S).
```

This rule states that a shelf cannot be put down by twice in the same time step. This rule applies to two different robots having the action putDownShelf at the same time for the same shelf, or one robot putting down the a shelf twice in the same time step.

Below are the following constraints I specified in the test stage:

- Robots can only do one action at a time step
- Robots cannot move off the grid
- 2 robots cannot pick up the same shelf at the same time
- Robots cannot pick up a shelf if it is carrying a shelf at the same time
- Robots cannot pickup the same shelf if its already being carried by another robot
- Robots cannot pickup a shelf if its not in the same cell as the shelf
- 2 robots cannot put down the same shelf at the same time
- Robots cannot put down a shelf if its not carrying a shelf
- Robots cannot put down a shelf on a highway
- Robots cannot deliver a shelf at a location if its not at the location
- Robots cannot deliver a shelf if it isn't carrying a shelf
- Robots cannot deliver more product than was ordered for
- Robots cannot deliver more product than the shelf held
- A picking station cannot exist at the same spot as a highway
- A shelf cannot exist at the same spot as a highway
- Robots cannot be in multiple cells at the same time
- Robots cannot be in the same cell as another robot at the same time
- Robots cannot swap locations in the same time step (no collisions)
- Shelves cannot be in two cells at the same time
- 2 shelves cannot be in the same cell at the same time
- Shelves cannot be on a cell floor (instead of a robot) and be carried at the same time

The final constraint is the goal constraint that specifies that all orders have to completed by the end of a set time period. The exact end point is specified by a constant value set by the programmer. This value is set so that when the valid solutions are trimmed down, solutions in which the robot wastes time unnecessarily moving around the grid are not included. As shown in Formula 6, my goal constraint uses n as the constant variable and by default it is set to twenty time steps.

Formula 6: The Goal Constraint

```
:- not orderAt(OI,object(cell,_),contains(PR,0),n), orderAt(OI,object(cell,_),contains(PR,_),0).
```

To find the most optimized action plan, I use two helper variables: timeTaken and actionAt. The actionAt variable keeps track of how many action each time step takes and timeTaken finds the latest time an action was taken. Using the minimize command in Clingo, as shown in Formula 7, I try to reduce the amount of time taken by an action plan.

Formula 7: The Optimization Step

```
#minimize { M@2 : timeTaken(M) }.
```

As a tiebreaker, I also minimize the number of total actions taken, which was not needed in the test instances.

Analysis of Results

The program I created was able to run on all of the test instances provided by Gebser and Obermeier. The maximum amount of time for all of the minimum solutions was instance 2 taking 11 time steps and 17 actions. The maximum number of steps for all of the minimum solutions was instance 1 taking 10 time steps but 19 actions. The minimum time for any of the solutions was instance 4, which only took 5 time steps to solve.

With ASP, the most amount of CPU time required to solve any instance was 3.3 seconds for the first instance. With the speed of receiving these results and the program I wrote being only 126 lines long, the scenario illustrated that it would not be a burden to use answer set programming in the real world to diagnosis and relay the correct paths the fleet of robots should take to delivery every order.

```
(py3.6) C:\Users\User\Downloads\automated-warehouse-scenario>clingo
inst3.asp solver.asp --quiet=1
clingo version 5.6.2
Reading from inst3.asp ...
solver.asp:118:13-14: info: tuple ignored:
  #inf@2

Solving...
Answer: 32
occurs(object(robot,1),move(-1,0),0) occurs(object(robot,1),move(0,-
1),1) occurs(object(robot,1),move(0,-1),3) occurs(object(robot,2),mo
ve(0,-1),3) occurs(object(robot,1),move(1,0),5) occurs(object(robot,
2),move(1,0),5) occurs(object(robot,1),pickup,2) occurs(object(robot
,2),pickup,2) occurs(object(robot,1),deliver(2,4,1),4) occurs(object
(robot,2),deliver(1,2,1),6) numActions(10) time_taken(7)
Optimization: 7 10
OPTIMUM FOUND

Models       : 32
  Optimum    : yes
Optimization : 7 10
Calls        : 1
Time         : 0.414s (Solving: 0.23s 1st Model: 0.01s Unsat: 0.01s)
CPU Time     : 0.406s
```

Figure 2: The output of running my solver program on the third test instance of the automated warehouse scenario. The third scenario can be solved in 10 steps over 7 time steps.

Conclusion

Through the Automated Warehouse Scenario, I showed a potential use case of KRR and ASP to be used in the real world to direct fleets of robots in an efficient way. The program I wrote was created using the structure of Generate-Define-Test, which helped to clearly specify the domain and limitations required to solve the Automated Warehouse Sc problem. The program was able to solve every test instance and was able to resolve each instance in under 3.5 seconds without parallelization. Due to the quick speed of compiling the program and small size of the program, I recommended that ASP should be considered for real world use to direct fleets of robots in automated warehouses.

Future Work

In order to make this solution more applicable to real world scenarios, the strict constraints on the robots and warehouse dimensions should be removed. Although there are many instances of flat robots moving shelves across factory floors, there are many other types of automated fetching robots that have different abilities including picking items off of a shelf instead of moving the full shelf. To make this use case more applicable to the real world, different floor plans should be considered when specifying the initial state of the warehouse. If differing floor plans are considered, the code to keep robots inside the grid would not function for non-rectangular shapes. Even though it is a useful oversimplification of the real world, testing should be done on the constraints needed for real world warehouses like the warehouses operating Amazon's Hercules robots (Amazon Staff, 2023).

References

Amazon Staff. "4 Cool Facts about Hercules, the Small-but-Mighty Robot in Amazon's Fulfillment Centers." Amazon News, September 7, 2023. https://www.aboutamazon.com/news/operations/amazon-hercules-robot.

Gebser, M., and Obermeier, P. 2019. ASP challenge problem: Automated warehouse scenario.

Lee, J. 2026. Practice of answer set programming. Lecture video series for Knowledge Representation and Reasoning course CSE 579.

Singhal, Amit. "Introducing the Knowledge Graph: Things, Not Strings." Google, May 16, 2012. https://blog.google/products-and-platforms/products/search/introducing-knowledge-graph-things-not/.